

# Artemis Student Platform (Eventer)

Документацията е актуализирана след имплементация на Event CRUD, Async Worker и системата за нотификации.

## 1. Обобщение: маршрути, таблици и функционалности

### Рути (всички в app.py, blueprint pages)

| Метод  | Път                          | Автентикация       | Отговор       | Поведение                             |
|--------|------------------------------|--------------------|---------------|---------------------------------------|
| GET    | /                            | Няма               | HTML          | Начална страница                      |
| GET    | /register                    | Няма               | HTML          | Форма за регистрация                  |
| POST   | /register                    | Няма               | HTML или JSON | Създава потребител; JSON 201 + JWT    |
| GET    | /login                       | Няма               | HTML          | Форма за вход                         |
| POST   | /login                       | Няма               | HTML или JSON | Проверява данните; session; JWT       |
| GET    | /api/me                      | Bearer JWT         | JSON          | Връща { id, email, role }             |
| GET    | /api/admin                   | Bearer JWT + admin | JSON          | Само admin; 403 за student            |
| GET    | /logout                      | Няма               | Redirect      | Изчиства session; пренасочва /        |
| GET    | /api/events                  | Bearer JWT         | JSON          | Списък с events + свободни места      |
| POST   | /api/events                  | Bearer JWT + admin | JSON          | Създава event (title, capacity)       |
| POST   | /api/events/<id>/register    | Bearer JWT         | JSON          | Регистрация: confirmed или waitlisted |
| DELETE | /api/events/<id>/register    | Bearer JWT         | JSON          | Отказ; промотира waitlist             |
| GET    | /api/notifications           | Bearer JWT         | JSON          | In-app inbox на потребителя           |
| PATCH  | /api/notifications/<id>/read | Bearer JWT         | JSON          | Маркира нотификация като прочетена    |
| GET    | /about                       | Няма               | HTML          | About страница                        |

JSON vs HTML: request.is\_json unu Accept: application/json header.

```
def wants_json_response():
    return request.is_json or "application/json" in request.headers.get("Accept", "")
App.py Lines 44-46
```

### База данни

| Файл              | Таблица            | Колони                                                                                                     |
|-------------------|--------------------|------------------------------------------------------------------------------------------------------------|
| users.db (SQLite) | users              | id, email (UNIQUE), password_hash, role (default 'student'), created_at                                    |
|                   | events             | id, title, description, capacity, created_at                                                               |
|                   | registrations      | id, user_id, event_id, status (confirmed waitlisted cancelled), waitlist_position, registered_at           |
|                   | notification_queue | id, event_type, user_id, payload (JSON), status (pending processing done failed), created_at, processed_at |
|                   | notifications      | id, user_id, event_type, title, body, read, created_at                                                     |

Миграционна логика -- ALTER TABLE за колони, добавени в по-стара база.

```
CREATE TABLE IF NOT EXISTS events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL, description TEXT, capacity INTEGER NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE IF NOT EXISTS registrations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL REFERENCES users(id),
    event_id INTEGER NOT NULL REFERENCES events(id),
    status TEXT NOT NULL DEFAULT 'confirmed', -- confirmed | waitlisted | cancelled
```

```
waitlist_position INTEGER,
registered_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
UNIQUE(user_id, event_id)
);
CREATE TABLE IF NOT EXISTS notification_queue (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event_type TEXT NOT NULL, user_id INTEGER NOT NULL REFERENCES users(id),
    payload TEXT NOT NULL, -- JSON
    status TEXT NOT NULL DEFAULT 'pending', -- pending | processing | done | failed
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, processed_at TEXT
);
CREATE TABLE IF NOT EXISTS notifications (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL REFERENCES users(id),
    event_type TEXT NOT NULL, title TEXT NOT NULL, body TEXT NOT NULL,
    read INTEGER NOT NULL DEFAULT 0,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

## Работещи функционалности

| Функционалност           | Статус      | Бележки                                        |
|--------------------------|-------------|------------------------------------------------|
| Регистрация (форма)      | Работи      | Email + парола; мин. 8 символа; regex за email |
| Регистрация (JSON)       | Работи      | Връща token, role; опционално поле role        |
| Вход (форма)             | Работи      | Задава session['user_id'], session['role']     |
| Вход (JSON)              | Работи      | Връща JWT (валидност 60 мин.)                  |
| Защитен /api/me          | Работи      | Изисква Bearer token                           |
| Защитен /api/admin       | Работи      | Само роля admin; 403 за student                |
| Изход (session)          | Работи      | Изчиства Flask session (JWT не се анулира)     |
| Хеширане на пароли       | Работи      | bcrypt                                         |
| Валидация на email       | Работи      | Regex на сървъра                               |
| Валидация на парола      | Работи      | Минимум 8 символа                              |
| Защита на session        | Частично    | HttpOnly, SameSite=Lax, Secure в production    |
| Event CRUD               | Работи      | GET/POST /api/events; register; cancel         |
| Waitlist управление      | Работи      | Auto-промоция при отказ от confirmed място     |
| Нотификации (in-app)     | Работи      | Inbox + mark as read                           |
| Async Worker (worker.py) | Работи      | Отделен процес; SQLite queue; WAL mode         |
| Превключване EN/BG       | Само клиент | script.js                                      |
| Начална страница         | Работи      | Статичен hero + bubble анимация                |

## Само UI / не е имплементирано

| UI елемент                             | Backend                                                  |
|----------------------------------------|----------------------------------------------------------|
| Навигация Clubs (href="#")             | Няма                                                     |
| name, class във формата за регистрация | Събират се в HTML, не се записват                        |
| Показване на грешка при вход           | Сървърът подава error, но login.html няма {% if error %} |

## Зависимости (без requirements.txt)

- flask

- bcrypt
- PyJWT (import jwt)
- sqlite3 (stdlib)
- json (stdlib)

## 2. Async Worker и Нотификации

### Архитектура на pipeline-a

Системата е изградена изцяло върху SQLite без external message broker. Два независими процеса споделят базата чрез WAL mode:

```
[HTTP заявка]
    |
Flask app.py --пише--> notification_queue (status='pending')
                                |
worker.py <--poll на 5s-----
    |
    +--пише--> notifications (in-app inbox)
```

### Feature Definitions

#### RegistrationConfirmed

**Trigger:** POST /api/events/<id>/register -- събитието има свободно място

**Action:** status='confirmed'; enqueue job payload: { event\_title }

**Notification:** *"You are confirmed for {event\_title}!"*

#### RegistrationWaitlisted

**Trigger:** POST /api/events/<id>/register -- събитието е пълно

**Action:** status='waitlisted'; waitlist\_position = следваща позиция

enqueue job payload: { event\_title, waitlist\_position }

**Notification:** *"You are on the waitlist for {event\_title} (position #{waitlist\_position})."*

#### WaitlistPromoted

**Trigger:** DELETE /api/events/<id>/register -- потребител отказва confirmed място

**Action:** Първият waitlisted -> confirmed; позициите се намаляват

enqueue job payload: { event\_title }

**Notification:** *"Great news! You have been moved off the waitlist for {event\_title}."*

### Background Worker (worker.py)

Worker-ът е отделен Python процес, стартиран независимо от Flask сървъра:

```
# Терминал 1
python3 app.py

# Терминал 2
python3 worker.py
```

Поведение:

- Активира PRAGMA journal\_mode=WAL при стартиране (concurrent SQLite достъп)
- Poll на notification\_queue на всеки 5 секунди (batch: 10 job-a)
- Маркира job като 'processing' преди обработка (предотвратява дублиране)
- При успех: пише в notifications, маркира 'done'
- При грешка: маркира 'failed', продължава с следващия
- Логва всичко в stdout

## 3. Задължителна текстова документация

### 3.1 Тестов план (стратегия W1 & W2)

#### Обхват

В обхвата (имплементирано):

- Автентикация чрез форма (/register, /login, /logout)
- JSON автентикация (POST /register, POST /login)
- JWT жизнен цикъл (издаване, употреба, изтичане, невалиден token)
- RBAC (/api/me, /api/admin)
- Валидация на вход (email, дължина, дублиран email)
- Event CRUD (GET/POST /api/events)
- Регистрация: confirmed и waitlisted
- Waitlist промоция при отказ
- In-app нотификации (inbox, mark as read)
- Async worker (worker.py)

Извън обхвата:

- Clubs страница
- Token refresh / revoke
- Автоматизиран тестов suite
- Email нотификации (SMTP)

#### W1 -- Настройка, автентикация и Postman основа

| Ден | Дейност                                                               | Резултат                             |
|-----|-----------------------------------------------------------------------|--------------------------------------|
| 1   | Инсталиране (flask, bcrypt, PyJWT); SECRET_KEY; стартиране            | Работещ сървър http://127.0.0.1:5000 |
| 2   | Smoke тестове: GET /, /register, /login; static файлове               | Smoke checklist                      |
| 3   | Happy path: форма register -> login -> logout; JSON register -> login | Дневник pass/fail                    |
| 4   | JWT поток: /api/me; липсващ/невалиден token                           | JWT дневник                          |
| 5   | Postman колекция + environment; експорт                               | Postman v1                           |

Критерии за W1: Всички auth маршрути работят; JWT се издава и приема на /api/me.

#### W2 -- Events, Notifications, Worker и регресия

| Ден | Дейност                                                                 | Резултат         |
|-----|-------------------------------------------------------------------------|------------------|
| 1   | RBAC: student vs admin на /api/admin                                    | RBAC дневник     |
| 2   | Event CRUD: create event (admin), list, register (confirmed/waitlisted) | Event дневник    |
| 3   | Waitlist: cancel confirmed -> WaitlistPromoted; inbox нотификации       | Waitlist дневник |
| 4   | Worker: стартиране, обработка на jobs; mark as read                     | Worker checklist |
| 5   | Security checklist; пълна регресия + финален sign-off                   | Тестов доклад    |

Критерии за W2: Events, waitlist и нотификации end-to-end; worker обработва всички типове jobs.

#### Тестова среда

| Променлива  | Препоръчителна стойност                    |
|-------------|--------------------------------------------|
| base_url    | http://127.0.0.1:5000                      |
| SECRET_KEY  | Силен случаен низ (избягвайте dev default) |
| FLASK_DEBUG | 0 (освен при debug)                        |

|                |                                 |
|----------------|---------------------------------|
| FLASK_ENV      | production (Secure cookie флаг) |
| JWT валидность | 60 минути (JWT_EXPIRY_MINUTES)  |

## 3.2 Postman workspace и environment конфигурация

### Environment: Eventer Local

| Променлива       | Начална стойност      | Описание                  |
|------------------|-----------------------|---------------------------|
| base_url         | http://127.0.0.1:5000 | Flask сървър              |
| student_email    | student@test.edu      | Student акаунт            |
| student_password | TestPass123!          | Мин. 8 символа            |
| admin_email      | admin@test.edu        | Admin акаунт              |
| admin_password   | AdminPass123!         | Admin парола              |
| auth_token       | (празно)              | От login/register отговор |
| student_token    | (празно)              | Student JWT               |
| admin_token      | (празно)              | Admin JWT                 |

### Структура на колекцията

```
Eventer - Artemis Platform
Auth - JSON API
  POST Register (Student / Duplicate -> 409 / Short Password -> 400 / Invalid Email -> 400)
  POST Login (Student) / POST Login (Invalid -> 401) / GET Logout
Auth - Protected Routes
  GET /api/me (Valid / No Token -> 401 / Invalid Token -> 401)
  GET /api/admin (Student -> 403 / Admin -> 200)
Auth - HTML Forms
  GET Register Form / POST Register (Form) / GET Login Form / POST Login (Form + Cookie)
Events
  POST /api/events (Admin token)
  GET /api/events (Student token)
  POST /api/events/<id>/register --> RegistrationConfirmed
  POST /api/events/<id>/register --> RegistrationWaitlisted (full event)
  DELETE /api/events/<id>/register --> WaitlistPromoted
Notifications
  GET /api/notifications
  PATCH /api/notifications/<id>/read
```

### Примерни заявки

#### POST Register (JSON)

```
POST {{base_url}}/register
Content-Type: application/json

{"email": "{{student_email}}", "password": "{{student_password}}", "role": "student"}
```

Tests: `pm.environment.set("student_token", pm.response.json().token);`

#### POST /api/events (Admin)

```
POST {{base_url}}/api/events
Authorization: Bearer {{admin_token}}
Content-Type: application/json

{"title": "Python Workshop", "description": "Intro to Flask", "capacity": 1}
```

#### POST /api/events/<id>/register

```
POST {{base_url}}/api/events/1/register
Authorization: Bearer {{student_token}}
```

Очаквано: 201 {"status": "confirmed"} или {"status": "waitlisted", "position": N}

Предварителна стъпка: създаване на admin

```
{"email": "{{admin_email}}", "password": "{{admin_password}}", "role": "admin"}
```



### 3.3 Auth тестови случаи (15+)

| ID      | Заглавие                    | Стъпки                                  | Очакван резултат                   |
|---------|-----------------------------|-----------------------------------------|------------------------------------|
| AUTH-01 | JSON регистрация (student)  | POST /register JSON { email, password } | 201; token, role: student          |
| AUTH-02 | JSON дублиран email         | Регистрация с един email два пъти       | 409; Email already exists.         |
| AUTH-03 | JSON кратка парола          | POST с парола < 8 символа               | 400; грешка за дължина             |
| AUTH-04 | JSON невалиден email        | POST email: not-an-email                | 400; грешка за email               |
| AUTH-05 | JSON успешен вход           | POST /login с валидни данни             | 200; { token, role }               |
| AUTH-06 | JSON неуспешен вход         | Грешна парола                           | 401; Invalid email or password.    |
| AUTH-07 | GET /api/me валиден token   | Bearer token от login                   | 200; { id, email, role }           |
| AUTH-08 | GET /api/me без token       | Без Authorization header                | 401; Missing bearer token.         |
| AUTH-09 | GET /api/me невалиден token | Bearer invalid.token.here               | 401; Invalid token.                |
| AUTH-10 | GET /api/admin като student | Student JWT                             | 403; insufficient role.            |
| AUTH-11 | GET /api/admin като admin   | Admin JWT                               | 200; Admin access granted.         |
| AUTH-12 | Form register + redirect    | POST form /register                     | 302 -> /login                      |
| AUTH-13 | Form login задава session   | POST form /login                        | 302 -> /; session cookie           |
| AUTH-14 | Logout изчиства session     | GET /logout след form login             | Session изчистен; redirect /       |
| AUTH-15 | Self-register като admin    | JSON register role: admin               | 201; admin роля (известен проблем) |

### 3.4 Event CRUD тестови случаи

Статус: ИЗПЪЛНИМИ -- backend е имплементиран в app.py.

| ID     | Заглавие                   | Стъпки                                           | Очакван резултат                         |
|--------|----------------------------|--------------------------------------------------|------------------------------------------|
| EVT-01 | Създаване на event (admin) | POST /api/events {title, capacity:1} admin token | 201; { id, title, capacity }             |
| EVT-02 | Списък с events            | GET /api/events с student token                  | 200; масив + available места             |
| EVT-03 | Регистрация (confirmed)    | POST /api/events/1/register -- свободно място    | 201; { status: confirmed }; job в queue  |
| EVT-04 | Регистрация (waitlisted)   | POST /api/events/1/register -- event пълен       | 201; { status: waitlisted, position: 1 } |
| EVT-05 | Дублирана регистрация      | POST register два пъти за един event             | 409; Already registered.                 |
| EVT-06 | Отказ от confirmed място   | DELETE /api/events/1/register (потребител А)     | 200; WaitlistPromoted job за Б           |
| EVT-07 | Event не съществува        | POST /api/events/999/register                    | 404; Event not found.                    |
| EVT-08 | Без admin token            | POST /api/events с student token                 | 403; insufficient role.                  |
| EVT-09 | Capacity = 0               | POST /api/events { capacity: 0 }                 | 400; capacity must be positive           |
| EVT-10 | Липсва title               | POST /api/events {}                              | 400; title is required                   |

### 3.5 Notification тестови случаи

| ID       | Заглавие                   | Стъпки                                         | Очакван резултат                            |
|----------|----------------------------|------------------------------------------------|---------------------------------------------|
| NOTIF-01 | RegistrationConfirmed job  | Регистрация при свободно място; изчакай worker | notifications: title=Registration Confirmed |
| NOTIF-02 | RegistrationWaitlisted job | Регистрация при пълен event; изчакай worker    | notifications: title=Added to Waitlist      |
| NOTIF-03 | WaitlistPromoted job       | Cancel confirmed; worker обработка             | notifications за Б: Promoted from Waitlist  |
| NOTIF-04 | GET /api/notifications     | Bearer token след обработка от worker          | 200; масив с нотификации                    |
| NOTIF-05 | Mark as read               | PATCH /api/notifications/1/read                | 200; Marked as read.                        |
| NOTIF-06 | Чужда нотификация          | PATCH с токен на друг потребител               | 404; Notification not found.                |
| NOTIF-07 | Worker batch               | 10 pending jobs; стартирай worker              | Всички -> done в 5 сек.                     |
| NOTIF-08 | Worker failed job          | Невалиден event_type в queue                   | Job -> failed; worker продължава            |

### 3.6 Security checklist (документация)

#### Автентикация и токени

| Проверка                           | Статус | Находка                                               |
|------------------------------------|--------|-------------------------------------------------------|
| Хеширани пароли (bcrypt)           | PASS   | hash_password() / verify_password()                   |
| Parameterized SQL                  | PASS   | ? placeholders навсякъде                              |
| JWT подпис (HS256)                 | PASS   | 60-минутна валидност                                  |
| Валидация на email формат          | PASS   | EMAIL_RE regex                                        |
| Мин. дължина на парола (8)         | PASS   | Server-side при register                              |
| Bearer token на защитени маршрути  | PASS   | /api/me, /api/admin, events, notifications            |
| Role check на admin маршрут        | PASS   | roles_required('admin')                               |
| Self-assign admin при register     | FAIL   | JSON role: admin е позволен                           |
| JWT revocation при logout          | FAIL   | Logout изчиства само session; JWT валиден до изтичане |
| Rate limiting / brute-force защита | FAIL   | Неограничени login опити                              |
| CSRF на HTML форми                 | FAIL   | Няма CSRF tokens                                      |

#### Session и конфигурация

| Проверка              | Статус   | Находка                                        |
|-----------------------|----------|------------------------------------------------|
| Secret от environment | ЧАСТИЧНО | SECRET_KEY / JWT_SECRET_KEY; fallback dev-only |
| Session HttpOnly      | PASS     | SESSION_COOKIE_HTTPONLY = True                 |
| Session SameSite      | PASS     | Lax                                            |
| Session Secure (prod) | PASS     | При FLASK_ENV=production                       |
| Debug режим           | ПОДОБРЕН | debug само ако FLASK_DEBUG=1                   |
| requirements.txt      | FAIL     | Липсва                                         |

#### JWT payload

JWT съдържа: sub (user id), email, role, iat, exp. Потребителят се проверява от DB при всяка защитена заявка (изтрити потребители получават 401).

## 4. Липсваща функционалност

| Изискване                     | Налице?  | Пропуск                                        |
|-------------------------------|----------|------------------------------------------------|
| Event CRUD (4+ теста)         | ДА       | Имплементирано; тестови случаи в раздел 3.4    |
| Waitlist управление           | ДА       | RegistrationWaitlisted + WaitlistPromoted      |
| Нотификации (in-app)          | ДА       | notification_queue + notifications + worker.py |
| UI страници за events         | НЕ       | Само API; href='#' placeholder връзки          |
| Token refresh / revoke        | НЕ       | Само stateless JWT                             |
| Контрол на admin provisioning | НЕ       | Всеки може да се регистрира като admin         |
| Автоматизирани тестове        | НЕ       | Няма pytest/unittest suite                     |
| requirements.txt              | НЕ       | Ръчна инсталация                               |
| Login error UX                | ЧАСТИЧНО | login.html няма error блок                     |
| Запазване на profile полета   | НЕ       | name/class се отхвърлят                        |
| Email нотификации (SMTP)      | НЕ       | Само in-app; SMTP не е конфигуриран            |
| Clubs страница                | НЕ       | Само # placeholder                             |

## 5. Слабости в сигурността и пропуски в тестването

### Висок приоритет

1. Privilege escalation -- 'role': 'admin' се приема на публичен /register.
2. Няма JWT invalidation -- откраднат token работи до изтичане, дори след logout.
3. Dev secret по подразбиране -- слаб, ако SECRET\_KEY не е зададен в env.

### Среден приоритет

4. Няма CSRF защита на HTML форми.
5. Няма rate limiting на login/register.
6. Двоен auth модел -- session + JWT; HTML маршрутите не ползват JWT.
7. Несъответствие в register -- name/class не се записват.

### Пропуски в тестването

| Област                       | Пропуск                                             |
|------------------------------|-----------------------------------------------------|
| Event CRUD                   | Имплементирано; тестови случаи в раздел 3.4         |
| Изтекъл token                | Изисква 60+ мин. или mock на времето                |
| Token след изтриване на user | Тестуемо чрез DB delete + /api/me                   |
| Session vs JWT parity        | Form login не помага на JSON-only защитени маршрути |
| Worker при DB недостъпност   | Retry логика не е имплементирана                    |
| Automated CI                 | Липсва pytest/unittest suite                        |
| Reproducible dependencies    | Няма requirements.txt / lockfile                    |
| Email нотификации            | SMTP не е конфигуриран; само in-app                 |